ECS713P Social Network Simulator – Technical Report (200356086)

1 – Design Decisions

The system is built around two primary data structures:

User Data Type: Defined as a record containing a username, userId (Int) and isOnline (Bool). The userId is specifically used as an index for the MVar collection, allowing O(1) time complexity when updating statistics.

Message Data Type: Although the current simulation prints messages to the console, the 'message' type is structured with sender, receiver, and content fields. This ensures the system is extensible for future features like conversation threading.

2 – Mvar Usage and Thread Coordination

To manage states across 10 concurrent threads, I implemented a two-tier MVar strategy:

Global Message Counter: A single 'messageCounter :: MVar Int': shared MVar that tracks the total number of messages sent across all threads. Every thread must acquire this MVar to check if the 100 message limit has reached. This ensures mutual exclusion, where one thread can increment the total amount at a time.

User Message Counters: '[MVar Int]': Rather than one MVar to protect all user statistics, I used 10 separate MVars (one per user). The "Divide and conquer" approach means threads only compete when updating the same user's counter. E.g. a thread updating 'User 3' count doesn't interfere with a thread updating 'User 7' count. However it is worth noting although concurrency improves performance, it is at the cost of more memory usage.

3 – Problems & Solutions

Race condition: During development, I encountered a race condition where the simulation would occasionally send 101 or 102 messages. This occurred because threads were checking the counter, releasing it, and then sending a message. I solved this by ensuring a "check and increment" logic, ensuring no other thread can read a invalid count value before the increment is finalised

4 – Extension: Online/Offline Status

To exceed the specification, I implemented a presence system where users are randomly assigned an 'isOnline' status (using randomIO).

Implementation Detail: I distinguished between type-based initialisation using 'randomIO :: IO Bool' for a 50/50 status split) and range-based selection using 'randomRIO' for message and recipient selection

Impact on simulation: Offline users remain in the system and still receive messages (inbox simulation), but they do not initiate message threads.

Thread behaviour: When an offline user's thread changes, it releases the global lock and enters a longer 'threadDelay' of 2 seconds before retrying. This adds a layer of realism by simulating varying levels of network activity and user engagement.